

Digital Delivery

A buyer whose payment was confirmed can read the seller's file. Nobody else can — not by knowing the URL, the order id, an asset id, or another buyer's grant, and not by editing the page. Access is *derived* on the server from a session and a query, never accepted from a caller.

Branch **claude/create-app-repository-1cxsih**

Head **c4b0b42**

1 commit · 31 files

Tests **317 passed** · 24 browser

Mutations **12 of 12 caught**

1

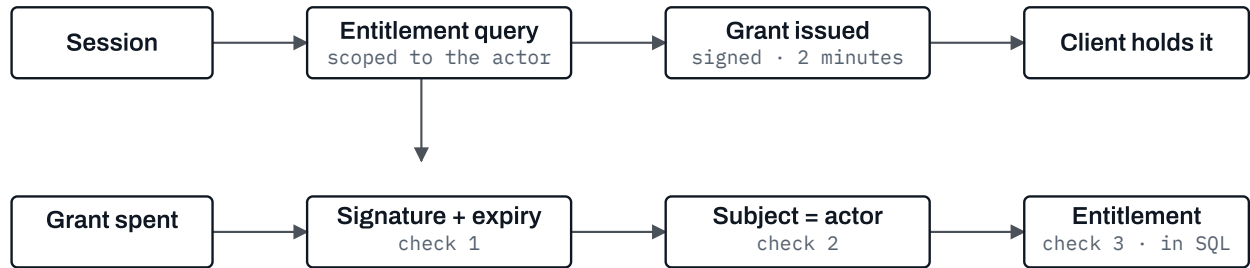
The entitlement and access architecture

ONE SENTENCE

Access is derived on the server from the authenticated actor plus a live entitlement, and from nothing the client says. No entitlement id, product id, order id, user id or "payment succeeded" flag sent by a caller is read as an authorization fact. The only thing a client supplies is *which asset it wants* — and even that is resolved through a query scoped to the actor.

The pieces, and who owns what

THING	WHAT IT IS
entitlements	The authority. One row per buyer per product, created only inside the transaction that marks an order paid. Carries <code>revoked_at</code> , so ending access is one statement.
digital_assets	The files. One product, many assets. <code>storage_key</code> is opaque, generated from uuids, and never leaves the server.
ContentStorage	A port. <code>put</code> and <code>open</code> take keys and return bytes — never a URL, so no code path can hand a client something that reads bytes without passing the authorization first.
content grant	A signed, two-minute hand-off. Carries no permission. It says only "Afrinext issued this, to this person, for this asset, and it has not expired".



THE GRANT IS A HAND-OFF, NOT AN AUTHORITY

The entitlement is read again when the grant is spent – so a revocation inside those two minutes stops it immediately, and a takedown stops it on the next request.

Two round trips, six checks between them. The top row issues a grant; the bottom row spends it. **Nothing in the bottom row trusts anything from the top row** except the fact that Afrinext signed it.

How the bytes are prevented from being read

Serving one byte requires three independent checks. Each is sufficient on its own to refuse, and each is a mutation in section 6 that fails a test when removed.

1 The grant verifies, and has not expired

HMAC-SHA256, compared in constant time, over the claims. The key is derived from the application secret with HKDF and a distinct label — *not* the secret itself, and not the OTP key — so a grant signature cannot be replayed against any other subsystem. Two minutes of life. A test edits the subject inside a grant, keeps the signature, and requires the refusal.

2 The session actor is the person the grant was issued to

So a grant copied out of one browser and pasted into another is refused *even when both people own the product* — proved by a test where both do. A grant with no session never reaches the domain at all: the route resolves an actor first, and answers `401`.

3 The entitlement is read again, from the database

One query joins asset → product → store → entitlement **for this actor**, and requires the entitlement to be live and both the product and its store to still be published. Another buyer's asset id, a draft product's asset id and an id that does not exist all resolve to nothing, and all produce the same refusal.

THE NINE CASES, AND WHERE EACH IS PROVED

Buyer with an entitlement can read · buyer without cannot · unauthenticated cannot · buyer A cannot use buyer B's grant · nobody can read an unpublished product's content · a seller cannot read another seller's file · a revoked entitlement grants nothing · changing the id in the request changes nothing · removing the button from the page changes nothing.

All nine are domain tests, and the last four are *also* browser tests: the unauthorized buyer signs in with their own real account and tries the library page, the grant endpoint, the buyer's own grant URL, and the same request with no session at all.

What the response says, and does not say

Every authenticated refusal is the same `403` with the same message. Expired grant, wrong subject, no entitlement, revoked, unpublished, no such asset — indistinguishable, so a prober learns nothing; the real reason goes to the audit log. A request with no session is a `401`, which is deliberate and leaks nothing: the caller already knows they are signed out, and the client needs to send them to sign in.

The bytes themselves are served `private, no-store` with `nosniff`, a `default-src 'none'; sandbox` policy and no referrer, and the filename is stripped before it enters a header. Uploads are restricted to an

allow-list that excludes `text/html` — these bytes come back from an Afrinext origin, so an accepted HTML upload would be stored XSS against our own domain.

The content model, and what it deliberately is not

`digital_assets` is one product to many files: title, kind, content type, size, SHA-256, opaque storage key, sort order. That is the minimum a PDF, a guide, a downloadable file or a protected resource needs.

ROOM FOR A COURSE, WITHOUT A COURSE

A future `Course → Module → Lesson` hangs its video and document resources off a table shaped like this one, rather than needing a parallel storage and access model. What this milestone does **not** build is that structure: there is no module, no lesson, no ordering beyond a sort key, no progress, no player, and `kind` is deliberately coarse — `document` and `file`. Adding `video` as a word here would imply a transcoder and a player that do not exist.

Storage

`ContentStorage` is a port with three methods and no concept of a URL. Two implementations ship: an in-memory one the domain tests run against, and a filesystem one used by the app and CI. **No object store is configured for Afrinext and none is invented.** The filesystem store is described in code as what it is — single-machine storage, right for development, CI and one server, wrong for two web instances behind a load balancer — and an S3-compatible adapter is a new class in that one file rather than a change anywhere else.

Keys are generated from uuids and never composed from client input. They are still validated: a shape check plus a resolved-path assertion, because a filesystem implementation joins a key onto a root directory and `../../etc/passwd` is the oldest bug there is. A test tries five malformed keys and a traversal against the real filesystem store.

Download policy, stated honestly

`products.delivery_mode` is `download` or `view_only`. A view-only product refuses an attachment grant at issuance — refused rather than quietly downgraded, so the seller's policy is a decision the server makes rather than a header the client picks — and is served `inline`.

THIS IS NOT DRM, AND NOTHING IN THE CODEBASE SAYS IT IS

`view_only` is a **policy about what Afrinext offers**, not a technical protection. The bytes are served, so anyone who can read them can save them. The comment in the schema says so, the comment in the domain says so, and the test that covers it says so. Calling browser-only viewing "protected" would be a claim the system cannot keep.

What *is* enforced is the entitlement: a view-only product still requires the same three checks, and a downloadable one is no less protected — **downloadable does not mean public**.

Entitlement is still payment's job

Nothing about entitlement creation changed. It still happens only inside the transaction that marks an order paid, is still idempotent against a unique index, and is still one row per buyer per product. What this milestone adds is tests that assert it *from the delivery side* — the side that now has something to lose:

SITUATION	ENTITLEMENT	CONTENT
Checkout opened, nothing paid	none	REFUSED
Payment initiated, still pending	none	REFUSED
Five concurrent confirmations of one event	exactly one	GRANTED ONCE
Order expired, provider confirms afterwards	none	REFUSED
Entitlement revoked	revoked, not deleted	REFUSED, MID-GRANT

THE EXPIRY CASE, RE-CHECKED AND NOT PAPERED OVER

The previous milestone flagged it: an order expires, the provider confirms afterwards, the payment succeeds, the order stays expired. A test now asserts from this side that **no entitlement is created and no content is reachable** — delivery does not quietly rescue the case.

It also does not invent a rule to resolve it. Deciding what happens to a buyer who was charged for an expired order *is* a refund and reconciliation decision, and that is yours to take. It remains the open risk it was.

Revocation

`revoked_at` is set, never deleted — the grant happened, and that it happened is part of why someone had access last month. **Nothing in this milestone revokes anything**, because deciding when access should end is a refund policy. The mechanism exists so that when that decision is taken it is a call site rather than a migration, and so that "a revoked entitlement grants nothing" is testable today. A test issues a grant, revokes, then spends the grant — and it fails, because the third check reads the database.

Twelve deliberate defects against the access-control guarantees. Each asserts its pattern was found before its result is believed.

DELIBERATE DEFECT	RESULT
The entitlement query stops scoping to the actor	CAUGHT
It stops requiring <code>revoked_at is null</code>	CAUGHT
It stops requiring the product and store to be published	CAUGHT · 2 TESTS
The subject check is removed — a grant becomes a bearer token	CAUGHT
The grant is trusted instead of re-reading the entitlement	CAUGHT
Grants stop expiring	CAUGHT
Grant signatures stop being verified	CAUGHT
Issuing a grant stops checking the entitlement	CAUGHT · 2 TESTS
<code>authorize()</code> is removed from asset upload	CAUGHT
The content-type allow-list is removed	CAUGHT
A view-only product accepts an attachment grant	CAUGHT
Storage keys stop being validated — traversal is possible	CAUGHT

THE FIRST ONE ESCAPED, AND IT IS THE MOST IMPORTANT ONE

Removing the actor scope from the entitlement query — the single check that makes access *per person* — changed nothing, and every test still passed. The reason is worth stating plainly: every "a stranger is refused" test used a product **nobody had bought**. So the request was refused because no entitlement existed at all, and a check that merely asked whether *some* entitlement existed would have passed the suite while granting the world access to anything with one buyer.

The test that catches it now gives the product **three buyers** first, then has a fourth person ask. That is the realistic shape of the attack — a popular product, and someone who did not pay — and it was not being tested at all.

```
pnpm lint                exit 0
pnpm typecheck            exit 0
pnpm test                 317 passed | 3 skipped (21 files)    was 284
pnpm build                exit 0
playwright test           24 passed                          was 22 · production mode, DB f
ledger concurrency        6 passed
reconcile-check           clean, zero drift
pnpm audit --audit-level high No known vulnerabilities found
mutation testing           12 of 12 caught
```

New: 33 domain tests in `content/content.test.ts` and 2 browser tests in `delivery.spec.ts`. No new dependency; `pnpm-lock.yaml` is untouched.

CI

CI, on this commit: run #20 (32394522332) on c4b0b42 — **ALL THREE JOBS GREEN**, 2 m 11 s.

Including a database built from zero, the schema rebuilt a second time to prove migration 0010 applies to an empty cluster, the ledger concurrency suite, the reconciliation check, and the browser suite against a production build.

THE BROWSER JOURNEY PROVES DELIVERY BY DELIVERY

The seller signs in, opens a store, creates a product, uploads a PDF through the seller's own form and publishes it. The buyer signs in, buys through the existing mock flow, and the confirmation arrives as a signed webhook from outside the browser. Then the buyer's page requests a grant and follows it — and the assertion is that **the HTTP response body equals the PDF the seller uploaded**. Nothing is declared successful by reading the database; SQL appears only to arrange the world.

31 files

FILE	WHY
The domain	
core/content/storage.ts	New. The port, key validation, and two implementations — in-memory and filesystem, the latter asserting every path stays under its root.
core/content/access.ts	New. Grant derivation and verification, the entitlement query, upload with its allow-list, the view-only rule, revocation.
core/content/content.test.ts	New. 33 tests, every one buying through the real payment path rather than inserting an entitlement.
core/content/index.ts · core/index.ts	Exports.
Schema	
db/migrations/0010 + schema	<code>digital_assets</code> , <code>products.delivery_mode</code> , <code>entitlements.revoked_at</code> , and a partial index for the live-entitlement lookup.
Authorization	
core/authz/permissions.ts	<code>product.manage_content</code> on <code>store_owner</code> . Scoped to the store, so one seller's grant says nothing about another's product.
Web	
api/v1/content/[token]/route.ts	New. The only route that returns protected bytes. Session first, then the grant, then the entitlement.
api/v1/library/.../access/route.ts	New. Mints a grant. Names an asset; never a user or an entitlement.
api/v1/products/[id]/assets/route.ts	New. The seller's upload and listing.
lib/content.ts	New. The storage instance and the grant key. The domain knows neither.
[locale]/library/... · AssetList.tsx	New. The buyer's screen. No durable href: clicking mints a grant and follows it.
sell/[storeSlug]/page.tsx · CatalogForms · catalog-actions	The seller's upload form, and a plain statement when a product has no file — a published product with nothing attached is one a buyer pays for and receives nothing from.
orders/page.tsx · product page	Owned products now link into the library rather than back to the sales page.
lib/api.ts	<code>content.forbidden</code> → 403.

FILE	WHY
e2e/delivery.spec.ts	New. The journey, then an unauthorized buyer trying five different ways.
core/auth/better-auth.ts	The verify rate rule — section 8.
e2e/global-setup.ts · playwright.config.ts · .env.example · .gitignore	Asset tables cleared between local runs; a per-run storage directory; the new variable documented.
core/test/harness.ts · i18n	The new table in the truncation list; 16 keys per locale.

CONFIRMATION	
No iPayMoney code, credentials, endpoints, payloads or signatures were added. The mock remains the only provider	CONFIRMED
No real settlement, no seller payout, no refund, no physical products, no riders, no services, no referral expansion, no course/video system	CONFIRMED
No ledger change. <code>ledger/</code> and <code>money/</code> have no diff, and entitlement creation posts nothing to the ledger	CONFIRMED
Payment confirmation and financial settlement remain separate, exactly as approved	CONFIRMED
Account consent and <code>seller_terms</code> enforcement untouched; no new consent system	CONFIRMED
The checkout, order and payment state machines, idempotency and webhook verification have no behavioural change	CONFIRMED
No new dependency	CONFIRMED

One change to approved code, reported rather than buried

THE /PHONE-OTP/VERIFY TRANSPORT RULE

It was a literal: twenty requests a minute per IP. This milestone's extra sign-ins tripped it, and four legitimate sign-ins were refused with a 429 *after* their codes had been sent.

That is the same defect the *send* rule was corrected for last milestone — a fixed per-IP cap that pre-empts the configured policy — and it is exactly what carrier NAT in Niamey looks like from the server's side. It is now derived from the stored policy, like the send rule. **Guessing is still bounded per challenge** by `max_attempts` in PostgreSQL, which is the control that stops brute force; this rule only ever bounded flood volume.

I introduced that literal myself in the checkout milestone. It is fixed here rather than left for the next suite to trip over.

Open risks

ITEM	STATE
The expired-order case is unchanged and unresolved. A buyer may have been charged for an order that expired; no entitlement, no content, no refund. Resolving it is a refund and reconciliation decision, and this milestone deliberately did not invent one.	NEEDS YOUR DECISION
Nothing revokes. The mechanism and its tests exist; no code path calls it, because when access should end is a refund policy.	AWAITING POLICY
Filesystem storage is single-machine. Correct for development, CI and one server; wrong for two web instances. Named as such in code. An object-store adapter is one new class implementing the port.	BEFORE SCALING OUT
Files are held in memory while being served. A 25 MiB cap keeps that bounded, but a streaming read is the right shape for larger assets and is what a video milestone would need.	WATCH
Nothing scans uploads. The allow-list stops HTML being served from our origin; it does not stop a malicious PDF. A seller can therefore distribute a hostile file to their own buyers.	OPEN
A grant is bound to a person, not to a device or an IP. Within its two minutes, someone who takes over that session could spend it — but so could they simply ask for a new one.	ACCEPTED
No versioning, replacement or deletion of assets; no per-asset ordering interface beyond the sort key; no download counting or audit-facing usage report beyond the access log.	OUT OF SCOPE

Proposed next, in the order I would take them: the refund and reconciliation decision that closes the expired-order case (it is the only item that can leave a real buyer out of pocket), then settlement — the first thing that should touch the ledger and the first that needs a real provider. **Not started.**